

Шпаргалка: 7 главных концепций аутентификации

Короткий справочник для backend-разработчиков: что не путать в Basic Auth, API Key, Bearer, JWT, session-based auth, OAuth2 и OpenID Connect.

Готово для Telegram, Notion, GitHub или PDF-публикации

Главная идея: эти термины относятся к разным слоям. Одни описывают способ передачи credentials, другие - формат токена, третьи - модель хранения сессии или протокол делегированного доступа.

Не складывайте Basic Auth, API Key, Bearer Token, JWT, Session, OAuth2 и OpenID Connect в один список как равнозначные “виды аутентификации”.

Содержание

- 1. Главная мысль видео
- 2. Authentication vs Authorization
- 3. Credentials
- 4. Basic Auth
- 5. API Key
- 6. Bearer Token и JWT
- 7. Session-based authentication
- 8. OAuth2
- 9. OpenID Connect
- 10. Главная таблица сравнения
- 11. Что нельзя смешивать
- 12. Короткие формулировки
- 13. Мини-проверка себя
- Что стоит дополнительно изучить

1. Главная мысль видео

Basic Auth, API Key, Bearer Token, JWT, Session, OAuth2 и OpenID Connect нельзя складывать в один список как равнозначные “виды аутентификации”.

Это сущности разных уровней:

Уровень	Примеры
HTTP-схема передачи credentials	Basic, Bearer
Credential / секрет / идентификатор доступа	password, API key, access token, session id
Формат токена	JWT
Модель хранения состояния входа	session-based auth
Фреймворк делегированного доступа	OAuth2
Identity layer поверх OAuth2	OpenID Connect

Главная ошибка: смотреть на список в инструментах вроде Postman и думать, что Basic Auth, Bearer Token, JWT, API Key и OAuth2 - это один тип сущностей.

Формула: сначала отделяем уровень термина, а уже потом сравниваем плюсы, минусы и сценарии применения.

2. Authentication vs Authorization

Термин	Вопрос	Смысл
Authentication	Кто ты? Докажи, что это ты.	Проверка личности или подлинности клиента.
Authorization	Что тебе разрешено?	Проверка доступа к действию или ресурсу.

Почему путают:

- оба процесса часто происходят рядом;
- оба могут использовать один и тот же токен;
- в интерфейсах инструментов всё часто лежит во вкладке Authorization;
- HTTP-заголовок называется Authorization, хотя в нём часто передают credentials для аутентификации.

Мини-пример:

Пользователь: Катя
Authentication: система проверила пароль / токен и поняла, что это Катя.
Authorization: система проверяет, может ли Катя открыть админку.

3. Credentials

Credentials - это данные, с помощью которых клиент доказывает системе свою личность, право доступа или владение секретом.

Credentials - это не всегда пароль. Это может быть ключ, токен, session id или client secret.

Credential	Что доказывает
username + password	Пользователь знает пароль.
API key	Клиент владеет выданным ключом.
access token	Клиент получил право обращаться к ресурсу.
session id	У клиента есть активная серверная сессия.
client secret	Приложение знает секрет клиента.

Простая формула:

Identification: “Я - alice@example.com”
Authentication: “Вот пароль / токен / ключ, который это доказывает”
Authorization: “Alice может читать заказы, но не может удалять пользователей”

4. Basic Auth

Basic Auth - HTTP-схема аутентификации, где клиент передаёт username:password, закодированные в Base64, в заголовке Authorization.

Формат:

```
Authorization: Basic <base64(username:password)>
```

Пример HTTP-запроса:

```
GET /admin HTTP/1.1
Host: example.com
Authorization: Basic YWxpY2U6c2VjcmV0
```

```
alice:secret -> Base64 -> YWxpY2U6c2VjcmV0
```

Важно:

- Base64 - это кодирование, не шифрование.
- Любой, кто увидит строку Base64, может декодировать её обратно.
- Basic Auth нужно использовать только поверх HTTPS.
- Логин и пароль уходят почти в каждом запросе.
- Для публичных web-приложений, SPA и мобильных клиентов это обычно плохой выбор.

5. API Key

API key - заранее выданный ключ, который клиент передаёт при каждом запросе к API.

Обычно API key идентифицирует не конкретный логин пользователя, а приложение, интеграцию, проект, технического клиента или аккаунт разработчика.

Где может передаваться:

- в отдельном header, например X-API-Key;
- в Authorization header;
- в query parameter;
- в cookie.

Пример через header:

```
GET /v1/reports HTTP/1.1
Host: api.example.com
X-API-Key: sk_live_123456
```

Пример через Authorization:

```
GET /v1/reports HTTP/1.1
Host: api.example.com
Authorization: ApiKey sk_live_123456
```

API Key	Пользовательский логин
Обычно выдаётся приложению или интеграции.	Принадлежит человеку.
Часто создаётся заранее.	Обычно вводится при логине.
Может жить долго.	Может быть частью интерактивного входа.
Сам по себе часто не содержит claims.	Логин указывает на identity пользователя.

Частая ошибка: считать API key полноценной заменой пользовательской аутентификации. Если ключ украли, злоумышленник сможет обращаться к API, пока ключ не отозван, не ограничен или не истёк.

6. Bearer Token и JWT

Здесь важно разделить два уровня:

```
Authorization: Bearer <token>
```

Часть	Что это
Authorization	HTTP-заголовок.
Bearer	Схема передачи токена.
<token>	Сам токен.
JWT	Возможный формат этого токена.

Bearer - это способ сказать: “доступ получает тот, кто предъявил этот токен”.

JWT - это формат токена: компактное представление claims, которое может быть подписано или зашифровано.

Пример Bearer + JWT:

```
GET /api/orders HTTP/1.1
Host: example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Разбор:

Bearer - схема передачи.
eyJhbGci... - токен.
JWT - формат токена, если токен состоит из header.payload.signature.

Почему фраза “JWT-авторизация” неточная:

- JWT не говорит, как токен передаётся.
- JWT не говорит, кто и по какому flow выдал токен.
- JWT не является отдельным протоколом авторизации.
- Авторизация может использовать claims из JWT, но сам JWT - это формат.

Лучше говорить:

Bearer authentication с JWT access token
API принимает access token в формате JWT через Authorization: Bearer

7. Session-based authentication

Session-based authentication - подход, при котором после логина сервер создаёт сессию и хранит состояние входа у себя.

Flow:

1. Пользователь вводит username/password.
2. Сервер проверяет credentials.
3. Сервер создаёт session id.
4. Сервер сохраняет сессию в session storage.
5. Сервер отправляет session id клиенту в cookie.
6. Браузер автоматически прикладывает cookie к следующим запросам.
7. Сервер по session id находит сессию.

Пример ответа после логина:

```
HTTP/1.1 200 OK
Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Lax
```

Следующий запрос:

```
GET /profile HTTP/1.1
Host: example.com
Cookie: sessionId=abc123
```

Что хранится на сервере:

sessionId -> userId, loginTime, expiresAt, roles/flags

Что хранится в cookie:

sessionId

Не весь пользователь. Не пароль. Не “вся сессия”. Обычно только идентификатор сессии.

Главное отличие от JWT:

Session-based auth	JWT-based token flow
Сервер хранит сессию.	Токен может быть самодостаточным.
По sessionId нужен lookup в storage.	Подпись JWT можно проверить локально.
Logout/revoke проще: удалить сессию.	Revoke сложнее, нужны blacklist/version/short TTL.
Хорошо для классических web apps.	Удобно для API, SPA, mobile, microservices.

8. OAuth2

OAuth2 - это фреймворк авторизации, а не “ещё один способ логина”.

Он отвечает на вопрос:

К каким ресурсам это приложение может получить доступ от имени пользователя?

Пример:

Вы даёте приложению доступ читать файлы в Google Drive.
Приложение получает access token.
С этим access token оно вызывает Google Drive API.

Что важно:

- OAuth2 - про delegated authorization.
- Пользователь не отдаёт пароль стороннему приложению.
- Пользователь выдаёт разрешение.
- Приложение получает access token.
- Access token доказывает не “кто пользователь”, а “какой доступ выдан клиенту”.

Access token в OAuth2-контексте:

credential, который клиент предъявляет resource server, чтобы получить доступ к protected resource.

Частая ошибка:

Неточно: “У нас логин через OAuth2”
Точнее: “У нас OAuth2 используется для получения access token”
Если речь именно про вход пользователя - обычно нужен OpenID Connect.

9. OpenID Connect

OpenID Connect / OIDC появился, чтобы добавить слой аутентификации поверх OAuth2.

OIDC отвечает на вопрос:

Кто этот пользователь?

Главная добавка OIDC:

ID token

ID token - это JWT, который содержит identity claims:

```
{
  "sub": "123456789",
  "email": "alice@example.com",
  "iss": "https://accounts.google.com",
  "aud": "my-client-id",
  "exp": 1730000000
}
```

Разница между access token и ID token:

Token	Для чего нужен
access token	Доступ к API / resource server.
ID token	Информация о том, кто пользователь.

Backend не должен просто декодировать ID token и верить содержимому. Нужно проверить подпись, iss, aud, exp и другие условия.

10. Главная таблица сравнения

Концепция	Что это такое	Уровень	Что передаётся по HTTP	Что часто путают
Basic Auth	HTTP-схема, где передаётся username:password в Base64.	HTTP authentication scheme	Authorization: Basic <base64>	Думают, что Base64 шифрует пароль.
API Key	Заранее выданный ключ для клиента/API-интеграции.	Credential / способ доступа к API	X-API-Key, query param, cookie или Authorization	Путают с логином пользователя.
Bearer Token	Схема предъявления токена: кто владеет токеном, тот получает доступ.	HTTP-схема / способ передачи токена	Authorization: Bearer <token>	Путают с JWT.
JWT	Формат токена с claims.	Формат токена	Может быть внутри Bearer <token>	Называют отдельным типом аутентификации.
Session-based auth	Сервер хранит сессию, клиент хранит session id.	Модель состояния входа	Cookie: sessionId=...	Путают session id с access token.
OAuth2	Фреймворк делегированной авторизации.	Authorization framework	Authorization code, access token, refresh token	Называют способом логина.
OpenID Connect	Identity layer поверх OAuth2.	Authentication / identity protocol	ID token, access token, authorization code	Путают с OAuth2 "в чистом виде".

11. Что нельзя смешивать

Authentication и authorization

Authentication проверяет, кто делает запрос. Authorization проверяет, что этому субъекту разрешено.

Способ передачи credentials и формат токена

Bearer говорит, как передать токен. JWT говорит, как токен устроен внутри.

Bearer и JWT

Bearer-токен может быть JWT, но не обязан. Это может быть opaque token - случайная строка без встроенных claims.

JWT и OAuth2

JWT - формат токена. OAuth2 - фреймворк получения и использования access token для делегированного доступа.

OAuth2 и OpenID Connect

OAuth2 сам по себе не даёт стандартного способа сказать приложению, кто пользователь. OIDC добавляет identity-слой и ID token.

Session id и access token

Session id - ключ к серверной сессии. Access token - credential для доступа к защищённому ресурсу.

API key и пользовательский логин

API key чаще идентифицирует приложение, интеграцию, проект или технического клиента. Логин идентифицирует человека.

12. Короткие формулировки, которые стоит запомнить

1. Authentication - кто ты. Authorization - что тебе можно.
2. Credentials - это не только пароль, а любые данные для доказательства доступа или личности.

3. Basic Auth передаёт username:password в Base64, а Base64 - не шифрование.
4. API key - это ключ доступа, а не обязательно логин пользователя.
5. Bearer говорит, как токен передаётся, но не говорит, какого он формата.
6. JWT - это формат токена, а не отдельный протокол авторизации.
7. Authorization: Bearer <token> - это не всегда JWT.
8. Session-based auth хранит состояние входа на сервере.
9. OAuth2 решает задачу делегированного доступа, а не просто логина.
10. OpenID Connect добавляет authentication поверх OAuth2 через ID token.

13. Мини-проверка себя

Вопрос: JWT и Bearer - это одно и то же?

Ответ: Нет. Bearer - схема передачи токена, JWT - возможный формат токена.

Вопрос: Почему "JWT-авторизация" - неточная фраза?

Ответ: Потому что JWT - формат. Авторизация - процесс проверки прав, который может использовать claims из JWT.

Вопрос: Что хранится в cookie при session-based auth?

Ответ: Обычно session id, а не пароль и не все данные пользователя.

Вопрос: Чем API key отличается от username/password?

Ответ: API key обычно выдаётся приложению или интеграции и не требует интерактивного логина пользователя.

Вопрос: OAuth2 - это authentication или authorization?

Ответ: В первую очередь authorization: делегированный доступ к ресурсам.

Вопрос: Что добавляет OpenID Connect к OAuth2?

Ответ: Identity layer и ID token, по которому клиент может понять, кто пользователь.

Вопрос: Почему Basic Auth нельзя использовать без HTTPS?

Ответ: Потому что username:password только закодированы в Base64, а не зашифрованы.

Что стоит дополнительно изучить

Эти темы есть рядом со сценарием, но их лучше вынести в отдельные материалы:

- Digest Auth - историческая HTTP-схема challenge-response, сейчас редко используется.
- Access token vs refresh token - зачем два токена и где безопасно хранить refresh token.
- HttpOnly, Secure, SameSite - базовая защита cookie.
- XSS и хранение токенов - почему localStorage опасен для refresh/access tokens.
- SSO - единый вход через identity provider.
- SAML vs OpenID Connect - enterprise XML-подход против современного OIDC/JWT-подхода.
- Scopes, roles, permissions - как моделировать authorization после authentication.